Ricerca lineare e binaria

Indice

Il problema della ricerca

Ricerca lineare

Ricerca binaria

Il problema della ricerca

Introduzione: Il problema della ricerca

- In questa sezione enunciamo con precisione il problema di cercare un valore in un array
- È una delle operazioni più frequenti eseguite da un calcolatore, ed è il punto di partenza dei due algoritmi del capitolo
- Fissiamo anche che cosa deve restituire un algoritmo quando il valore **non** è nell'array

Cercare un valore

- Cercare un valore in una collezione di dati è una delle operazioni più frequenti eseguite da un calcolatore:
  - un nome in una rubrica;
  - un codice in una tabella di prodotti;
  - una parola in un dizionario.

- In questo capitolo la collezione è un array di numeri naturali, e presentiamo due algoritmi che risolvono il problema:
  - la **ricerca lineare**, che funziona su ogni array;
  - la **ricerca binaria**, che funziona solo su array ordinati in modo non decrescente, ma che è molto più veloce della ricerca lineare su array lunghi.

Definizione 1: problema della ricerca

Dati un array $a[1..n]$ di n numeri naturali e un numero naturale m da cercare, il ***problema della ricerca*** *consiste nel determinare se m è uno degli elementi dell'array, cioè se esiste un indice $k \in \{1, 2, \ldots, n\}$ tale che*

$$a[k] = m.$$

Un algoritmo per il problema della ricerca restituisce tale indice k se esiste, e il valore -1 altrimenti.

Perché il valore -1?

- Il valore -1 non è mai un indice valido dell'array $a[1..n]$
- Non lo è nemmeno di un array C++, i cui indici partono da 0
- Per questo motivo può essere usato per segnalare che il valore cercato **non è stato trovato**

Riepilogo

Il problema della ricerca:

- Input: un array $a[1..n]$ di numeri naturali e un valore m da cercare
- Output: un indice k con $a[k] = m$, se esiste
- Altrimenti il valore -1, che non è mai un indice valido

Due algoritmi:

- la ricerca lineare, per ogni array
- la ricerca binaria, per array ordinati in modo non decrescente

Ricerca lineare

Introduzione: Ricerca lineare

- In questa sezione presentiamo l'algoritmo più semplice per il problema della ricerca: confrontare il valore cercato con gli elementi uno dopo l'altro
- Funziona su **ogni** array, senza alcuna ipotesi sull'ordine degli elementi
- Lo eseguiamo su un esempio e contiamo esattamente quanti confronti esegue

Il problema

Dati un array $a[1..n]$ di n numeri naturali
e un numero naturale m da cercare,
consideriamo il problema di determinare
se m è uno degli elementi dell'array.

- L'idea più semplice è confrontare il valore cercato m con gli elementi dell'array uno dopo l'altro, dal primo all'ultimo
- e fermarsi non appena si trova un elemento uguale a m.

Questo è lo pseudocodice dell'algoritmo di ricerca lineare (Linear Search)
che determina se un valore m è presente
in un array $a[1..n]$ di n numeri naturali.

Algorithm: LinearSearch

Input : Un array $a[1..n]$ di n numeri naturali e un valore m da cercare

Output : Indice di m in a, oppure -1 se non trovato

```
1  function LinearSearch(a, n, m)
2      i := 1
3      while i ≤ n do
4          if a[i] = m then
5              return i   // Elemento trovato all'indice i
6          end
7          i ← i + 1
8      end
9      return −1                    // Elemento non trovato
10 end
```

Esegui in Colab: Python · C++ · Java

Come funziona LinearSearch

- L'algoritmo scorre l'array sequenzialmente dall'indice 1 a n
- A ogni iterazione confronta l'elemento $a[i]$ con il valore cercato m (riga 4)
- Se $a[i] = m$, l'elemento è trovato e viene restituito l'indice i (riga 5)
- Se il ciclo termina senza trovare corrispondenze, l'elemento non è nell'array e viene restituito -1 (riga 9)

Se m compare più di una volta nell'array, l'algoritmo si ferma alla prima corrispondenza:
l'indice restituito è il **più piccolo** indice k tale che $a[k] = m$.

Esempio (1/2)

Consideriamo l'array $a = [4, 2, 7, 1, 5, 9, 3]$ con $n = 7$.

Ricerca di $m = 5$:

iterazione	i	$a[i]$	azione
1	1	4	$a[i] \neq m$
2	2	2	$a[i] \neq m$
3	3	7	$a[i] \neq m$
4	4	1	$a[i] \neq m$
5	5	5	$a[i] = m \Rightarrow$ Trovato!

Quindi, l'elemento 5 si trova all'indice 5.

Esempio (2/2)

Consideriamo l'array $a = [4, 2, 7, 1, 5, 9, 3]$ con $n = 7$. Ricerca di $m = 6$:

iterazione	i	$a[i]$	azione
1	1	4	$a[i] \neq m$
2	2	2	$a[i] \neq m$
3	3	7	$a[i] \neq m$
4	4	1	$a[i] \neq m$
5	5	5	$a[i] \neq m$
6	6	9	$a[i] \neq m$
7	7	3	$a[i] \neq m$
–	8	–	$i > n \Rightarrow$ il ciclo termina, non trovato!

Quindi, l'elemento 6 non è presente nell'array (restituisce -1).

Contare i confronti

- Il lavoro svolto dalla ricerca lineare si misura con il numero di confronti $a[i] = m$ alla riga 4
- Nell'esempio:
  - servono 5 confronti per trovare $m = 5$;
  - tutti gli $n = 7$ elementi vengono confrontati per concludere che $m = 6$ non è nell'array.
- L'osservazione seguente dà il numero esatto di confronti in generale

Osservazione 1: numero di confronti della ricerca lineare

Dati un array $a[1..n]$ di n numeri naturali e un valore m da cercare:

- *se k è il più piccolo indice tale che $a[k] = m$, allora `LinearSearch` esegue esattamente k confronti alla riga 4 e restituisce k;*
- *se m non è nell'array, allora `LinearSearch` esegue esattamente n confronti alla riga 4 e restituisce -1.*

Dimostrazione

La variabile i assume i valori $1, 2, 3, \ldots$ in quest'ordine, e per ogni valore $i \leq n$ viene eseguito esattamente un confronto $a[i] = m$.

- Se k è il più piccolo indice con $a[k] = m$, allora $a[i] \neq m$ per $i \in \{1, 2, \ldots, k-1\}$, quindi l'algoritmo non si ferma per questi valori di i; per $i = k$ il confronto è vero e l'algoritmo restituisce k. I confronti sono quelli con $i = 1, 2, \ldots, k$, cioè k confronti.

- Se m non è nell'array, ogni confronto è falso, quindi il ciclo viene eseguito per $i = 1, 2, \ldots, n$ e si ferma quando $i = n + 1$. I confronti sono n, e l'algoritmo restituisce -1 alla riga 9.

□

Pregio e prezzo della ricerca lineare

La ricerca lineare non fa **alcuna ipotesi**
sull'ordine degli elementi.

Il suo prezzo è che, quando il valore cercato è assente
(o si trova nell'ultima posizione),
tutti gli n elementi dell'array devono essere esaminati.

Riepilogo

Ricerca lineare:

- confronta m con $a[1], a[2], \ldots$ in quest'ordine e si ferma alla prima corrispondenza
- funziona su ogni array e restituisce il **più piccolo** indice k con $a[k] = m$

Numero di confronti:

- esattamente k se la prima occorrenza di m è all'indice k
- esattamente n se m non è nell'array

Ricerca binaria

Introduzione: Ricerca binaria

- In questa sezione l'array è **ordinato**, e sfruttiamo l'ordine per scartare metà dell'array a ogni passo
- Scriviamo lo pseudocodice della ricerca binaria, lo eseguiamo su un esempio e ne disegniamo gli intervalli di ricerca
- Infine limitiamo il suo numero di iterazioni: circa $\log_2 n$ invece di n

Cercare una parola in un dizionario

Pensiamo a come si cerca una parola in un dizionario cartaceo:
nessuno lo legge dalla prima pagina.

Lo apriamo a metà e,
poiché le parole sono in ordine alfabetico,
sappiamo in quale metà proseguire.

La ricerca binaria applica la stessa idea
a un array ordinato.

Definizione 2: array ordinato

Un array $a[1..n]$ *di* n *numeri naturali è* ***ordinato in modo non decrescente*** *se*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

Dati un array $a[1..n]$ di n numeri naturali
ordinato in modo non decrescente e un numero naturale m da cercare,
consideriamo il problema di determinare
se m è uno degli elementi dell'array.

L'intervallo di ricerca

- La ricerca binaria mantiene una porzione $a[left..right]$ dell'array, detta **intervallo di ricerca**, che è l'unica porzione che può ancora contenere m
- All'inizio l'intervallo di ricerca è l'intero array
- A ogni iterazione il valore cercato m viene confrontato con l'elemento $a[mid]$ al centro dell'intervallo di ricerca:
  - se $a[mid] < m$, allora, poiché l'array è ordinato, tutti gli elementi $a[left], \ldots, a[mid]$ sono minori di m, e la ricerca prosegue nella metà destra $a[mid + 1..right]$;
  - se $a[mid] > m$, allora tutti gli elementi $a[mid], \ldots, a[right]$ sono maggiori di m, e la ricerca prosegue nella metà sinistra $a[left..mid - 1]$.

Questo è lo pseudocodice dell'algoritmo di ricerca binaria (Binary Search)
che determina se un valore m è presente
in un array $a[1..n]$ di n numeri naturali
ordinato in modo non decrescente.

Algorithm: BinarySearch (1/2)

Input : Un array $a[1..n]$ di n numeri naturali ordinato in modo non decrescente e un valore m da cercare

Output : Indice di m in a, oppure -1 se non trovato

```
1 function BinarySearch(a, n, m)
2     left := 1                                  // Estremo sinistro
3     right := n                                 // Estremo destro
4     while left ≤ right do
5         mid := left + ⌊(right − left)/2⌋       // Indice centrale
6         if a[mid] = m then
7             return mid                         // Elemento trovato
8         end
```

Algorithm: BinarySearch (2/2)

```
function BinarySearch(a, n, m) (continua)
    while left ≤ right (continua) do
 9      if a[mid] < m then
10          left ← mid + 1   // Cerca nella metà destra
11      end
12      else
13          right ← mid − 1          // Cerca nella metà
                sinistra
14      end
15  end
16  return −1                    // Elemento non trovato
17 end
```

Esegui in Colab: Python · C++ · Java

Come funziona BinarySearch (1/2)

- L'algoritmo mantiene due indici, *left* e *right*, che definiscono l'intervallo di ricerca $a[left..right]$, il quale contiene $right - left + 1$ elementi
- A ogni iterazione calcola l'indice centrale *mid* (riga 5) e confronta $a[mid]$ con il valore cercato m
- Se $a[mid] = m$, l'elemento è trovato e ne viene restituito l'indice (riga 7)
- Se $a[mid] < m$, la ricerca prosegue nella metà destra aggiornando $left \leftarrow mid + 1$ (riga 10)
- Se $a[mid] > m$, la ricerca prosegue nella metà sinistra aggiornando $right \leftarrow mid - 1$ (riga 13)

Come funziona BinarySearch (2/2)

- Se l'intervallo di ricerca diventa vuoto ($left > right$), l'elemento non è nell'array e viene restituito -1 (riga 16)
- Poiché
$$0 \leq \lfloor (right - left)/2 \rfloor \leq right - left,$$
l'indice *mid* appartiene sempre all'intervallo di ricerca:
$$left \leq mid \leq right.$$

Esempio (1/4)

Consideriamo l'array ordinato $a = [3, 9, 11, 12, 12, 16, 19]$ con $n = 7$.

Ricerca di $m = 12$:

iterazione	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] = m \Rightarrow$ Trovato!

Quindi, l'elemento 12 si trova all'indice 4.

Esempio (2/4)

Consideriamo l'array ordinato $a = [3, 9, 11, 12, 12, 16, 19]$ con $n = 7$.

Ricerca di $m = 10$:

iterazione	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] > m \Rightarrow right \leftarrow 3$
2	1	3	2	9	$a[mid] < m \Rightarrow left \leftarrow 3$
3	3	3	3	11	$a[mid] > m \Rightarrow right \leftarrow 2$
–	3	2	–	–	$left > right \Rightarrow$ Non trovato!

Quindi, l'elemento 10 non è presente nell'array (restituisce -1).

Esempio (3/4)

Consideriamo l'array ordinato $a = [3, 9, 11, 12, 12, 16, 19]$ con $n = 7$.

Ricerca di $m = 3$:

iterazione	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] > m \Rightarrow right \leftarrow 3$
2	1	3	2	9	$a[mid] > m \Rightarrow right \leftarrow 1$
3	1	1	1	3	$a[mid] = m \Rightarrow$ Trovato!

Quindi, l'elemento 3 si trova all'indice 1.

Esempio (4/4)

Consideriamo l'array ordinato $a = [3, 9, 11, 12, 12, 16, 19]$ con $n = 7$.

Ricerca di $m = 19$:

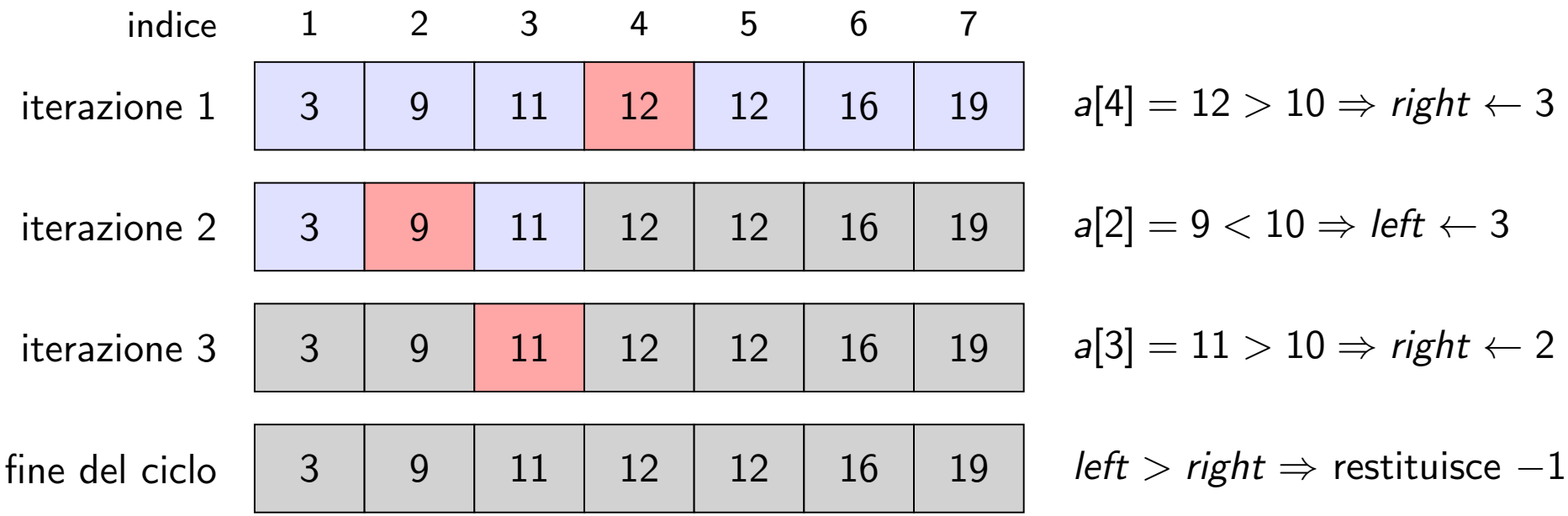
iterazione	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] < m \Rightarrow left \leftarrow 5$
2	5	7	6	16	$a[mid] < m \Rightarrow left \leftarrow 7$
3	7	7	7	19	$a[mid] = m \Rightarrow$ Trovato!

Quindi, l'elemento 19 si trova all'indice 7.

Gli intervalli di ricerca

- La figura della slide seguente mostra l'intervallo di ricerca a ogni iterazione della ricerca di $m = 10$
- Gli elementi fuori dall'intervallo di ricerca (grigi) sono scartati
- L'elemento $a[mid]$ confrontato con m è evidenziato in rosso

Gli intervalli di ricerca per $m = 10$



Intervalli di ricerca di `BinarySearch` per $m = 10$ in $a = [3, 9, 11, 12, 12, 16, 19]$: le celle blu formano l'intervallo di ricerca $a[left..right]$, la cella rossa è $a[mid]$, le celle grigie sono state scartate.

Valori ripetuti

- Se m compare più di una volta nell'array, la ricerca binaria restituisce l'indice di **una** delle sue occorrenze, non necessariamente la prima

- Per esempio, per $a = [1, 2, 2, 2, 2, 2, 3]$ e $m = 2$:
  - la prima iterazione calcola $mid = 1 + \lfloor 6/2 \rfloor = 4$;
  - poiché $a[4] = 2 = m$, l'algoritmo restituisce 4;
  - mentre `LinearSearch` restituisce 2.

Numero di iterazioni

- Ogni iterazione che non trova m scarta l'elemento $a[mid]$ e almeno metà dei restanti elementi dell'intervallo di ricerca
- Per questo la ricerca binaria è veloce: la lunghezza dell'intervallo di ricerca si (almeno) **dimezza** a ogni iterazione
- L'osservazione seguente trasforma questa idea in un limite al numero di iterazioni

Osservazione 2: numero di iterazioni della ricerca binaria

Dati un array $a[1..n]$ *di* $n \geq 1$ *numeri naturali ordinato in modo non decrescente e un valore* m *da cercare, il ciclo while di* `BinarySearch` *esegue al più*

$$\lfloor \log_2 n \rfloor + 1$$

iterazioni.

- Idea della dimostrazione: mostriamo che il numero di elementi dell'intervallo di ricerca almeno si dimezza a ogni iterazione che non restituisce

Dimostrazione (1/2)

Sia $L = \mathit{right} - \mathit{left} + 1 \geq 1$ il numero di elementi dell'intervallo di ricerca all'inizio di un'iterazione che non restituisce. Poiché $\mathit{mid} = \mathit{left} + \lfloor (L-1)/2 \rfloor$, il nuovo intervallo di ricerca ha:

- $\mathit{mid} - \mathit{left} = \lfloor (L-1)/2 \rfloor$ elementi, se $\mathit{right} \leftarrow \mathit{mid} - 1$;
- $\mathit{right} - \mathit{mid} = (L-1) - \lfloor (L-1)/2 \rfloor = \lceil (L-1)/2 \rceil$ elementi, se $\mathit{left} \leftarrow \mathit{mid} + 1$.

In entrambi i casi il nuovo numero di elementi è al più

$$\lceil (L-1)/2 \rceil = \lfloor L/2 \rfloor \leq L/2.$$

Dimostrazione (2/2)

All'inizio $L = n$. Quindi, dopo k iterazioni che non restituiscono, l'intervallo di ricerca ha al più $n/2^k$ elementi.

Un'iterazione viene eseguita solo se l'intervallo di ricerca non è vuoto, quindi l'iterazione $k + 1$ viene eseguita solo se

$$1 \leq \frac{n}{2^k}, \qquad \text{cioè} \qquad k \leq \log_2 n.$$

Poiché k è un intero, $k \leq \lfloor \log_2 n \rfloor$, e il numero $k + 1$ di una qualsiasi iterazione eseguita è al più $\lfloor \log_2 n \rfloor + 1$. □

Il limite viene raggiunto

- Nell'esempio precedente, $n = 7$ e

$$\lfloor \log_2 7 \rfloor + 1 = 3$$

- Le ricerche di $m = 10$, $m = 3$ e $m = 19$ eseguono esattamente 3 iterazioni

- Quindi il limite viene **raggiunto**

Ricerca lineare e ricerca binaria a confronto

- La differenza con la ricerca lineare cresce molto rapidamente con n
- La tabella confronta il numero di confronti della ricerca lineare quando m non è nell'array con il numero massimo di iterazioni della ricerca binaria:

n	ricerca lineare n confronti	ricerca binaria al più $\lfloor \log_2 n \rfloor + 1$ iterazioni
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

Quando usare la ricerca binaria

- La ricerca binaria richiede che l'array sia **ordinato** in modo non decrescente
- Su un array ordinato di n elementi esegue al più $\lfloor \log_2 n \rfloor + 1$ iterazioni, mentre la ricerca lineare può richiedere n confronti
- Se l'array non è ordinato, la ricerca binaria può scartare erroneamente la metà che contiene m, e si deve usare la ricerca lineare

Riepilogo (1/2)

Ricerca binaria:

- funziona su array ordinati in modo non decrescente
- mantiene l'intervallo di ricerca $a[left..right]$ e confronta m con il suo elemento centrale $a[mid]$
- prosegue nella metà destra se $a[mid] < m$, nella metà sinistra se $a[mid] > m$

Valori ripetuti:

- restituisce una delle occorrenze di m, non necessariamente la prima

Riepilogo (2/2)

Numero di iterazioni:

- ogni iterazione che non restituisce almeno dimezza l'intervallo di ricerca
- al più $\lfloor \log_2 n \rfloor + 1$ iterazioni, e il limite può essere raggiunto

Confronto con la ricerca lineare:

- 30 iterazioni invece di $1\,000\,000\,000$ confronti per $n = 1\,000\,000\,000$
- su un array non ordinato, solo la ricerca lineare è corretta

Riepilogo: punti chiave del capitolo

- Il problema della ricerca richiede un indice k tale che $a[k] = m$, oppure -1 se tale indice non esiste
- La ricerca lineare funziona su ogni array, restituisce la prima occorrenza di m ed esegue fino a n confronti
- La ricerca binaria funziona solo su array ordinati in modo non decrescente; dimezza l'intervallo di ricerca $a[left..right]$ a ogni iterazione ed esegue al più $\lfloor \log_2 n \rfloor + 1$ iterazioni
- Nello pseudocodice gli indici vanno da 1 a n, nei notebook Colab (Python, C++, Java) da 0 a $n - 1$: gli indici restituiti differiscono di 1